



UNIVERSIDAD
DE GRANADA

Práctica 3: Algoritmos Voraces

DAVID ÁVILA GIMÉNEZ

JOSÉ PORTILLO HARO

ALEJANDRO MORENO GARCÍA

NEREA LÓPEZ ROELAS

RAFAEL FERNANDO RIVAS CABRERA

Algorítmica

C3

1. Gestión editorial.....	3
1.1. Formulación del problema dentro del esquema greedy.....	3
1.2. Algoritmo voraz propuesto.....	4
1.3. Explicación del algoritmo.....	5
1.4. Análisis de eficiencia teórica.....	5
1.5. Demostración de optimalidad.....	6
1.6. Algoritmo alternativo de comparación.....	6
1.6.1. Eficiencia teórica del algoritmo alternativo.....	7
1.7. Implementación de los algoritmos.....	7
1.8. Análisis de eficiencia empírica.....	11
2. Cámaras de vigilancia.....	12
2.1. Explicación del Diseño y Análisis del Algoritmo Voraz.....	12
2.2. Análisis de las partes relevantes del código.....	13
2.3. Casos subóptimos: Dónde y por qué falla el algoritmo.....	14
2.3.1. Primer fallo.....	14
2.3.2. Segundo fallo.....	15
2.3.3. Tercer fallo.....	15
2.4. Eficiencia Teórica.....	16
2.5. Eficiencia Empírica.....	17
2.6. Eficiencia Híbrida.....	23
3. Cámaras en árbol (opcional).....	24
3.1. Diferencias en complejidad.....	24
3.2. Diseño del algoritmo.....	24
3.2.1. Explicación de la estrategia.....	24
3.2.2. Pseudocódigo.....	25
3.3. Conclusiones.....	25

ALGORITMOS VORACES

1. Gestión editorial

Una editorial científica debe gestionar la publicación de n artículos en una plataforma digital. Cada artículo tiene que pasar por dos fases consecutivas. En primer lugar, una revisión editorial centralizada, realizada por un único comité editorial, que solo puede revisar un artículo cada vez, y que requiere $P(i)$ unidades de tiempo para el artículo i . En segundo lugar, una fase de maquetación y publicación, que requiere $f(i)$ unidades de tiempo, y que puede realizarse en paralelo para todos los artículos, ya que la editorial dispone de suficientes maquetadores. El objetivo es decidir el orden en el que se revisan los artículos para minimizar el tiempo total hasta que el último artículo quede publicado.

En este caso la segunda fase no genera conflictos entre artículos, porque puede ejecutarse simultáneamente. Por ello, la única decisión realmente importante consiste en determinar el orden de la fase de revisión, ya que esa decisión marca el momento en el que cada artículo podrá comenzar su maquetación. Una vez un artículo termina la revisión, pasa inmediatamente a la segunda fase, pero el instante final en el que quedará completamente publicado dependerá tanto de lo que haya tardado en esperar su revisión como del tiempo de maquetación que tenga asociado.

Si llamamos $\pi=(\pi_1,\pi_2,\dots,\pi_n)$ al orden de revisión elegido, y definimos:

$$P_k=\sum_{r=1}^k p(\pi_r), \text{ para } r = 1 \text{ hasta } k$$

entonces el artículo situado en la posición k del orden terminará en el instante

$$C_k = P_k + f(\pi_k)$$

Por tanto, el tiempo total del proceso será

$$T(\pi) = \max_{1 \leq k \leq n} (P_k + f(\pi_k))$$

El problema consiste en encontrar un orden π que minimice ese valor máximo.

1.1. Formulación del problema dentro del esquema greedy

Siguiendo el esquema general de los algoritmos voraces, primero identificamos los distintos elementos del problema: un conjunto de candidatos, un conjunto de seleccionados, una función de selección, una función de factibilidad, una función solución y una función objetivo. Este tipo de algoritmos van construyendo la solución por etapas, tomando en cada paso la decisión local más prometedora.

En nuestro caso, estos elementos quedan definidos de la siguiente forma:

- **Conjunto de candidatos:** los artículos que todavía no han sido colocados en la secuencia de revisión.
- **Conjunto de seleccionados:** los artículos que ya han sido elegidos y ordenados dentro de dicha secuencia.
- **Función solución:** se alcanza una solución cuando todos los artículos han sido colocados en el orden de revisión.
- **Función de factibilidad:** cualquier artículo no seleccionado todavía puede añadirse al final de la secuencia, ya que no existe ninguna restricción de incompatibilidad entre artículos. Por tanto, todos los candidatos restantes son factibles.
- **Función objetivo:** minimizar el instante en el que se publica el último artículo, es decir, minimizar el valor de $\max_{1 \leq k \leq n} (P_k + f(\pi_k))$
- **Función de selección:** escoger en cada paso el artículo con mayor tiempo de maquetación $f(i)$ entre los que quedan sin revisar.

Queremos conseguir que si un artículo tiene una fase de maquetación muy larga, salga del comité editorial lo antes posible para que esa segunda fase empiece pronto. Si lo dejamos para el final, su tiempo de maquetación se sumará al retraso acumulado por todas las revisiones anteriores, aumentando el instante final del proceso. Entonces priorizar aquellos artículos cuyo tiempo de publicación posterior es mayor, para que esa fase larga se vaya ejecutando en paralelo cuanto antes.

1.2. Algoritmo voraz propuesto

El algoritmo voraz que proponemos consiste en **ordenar los artículos en orden no creciente de $f(i)$** , de mayor a menor tiempo de maquetación y publicación. Una vez fijado ese orden, se simula la revisión secuencial de los artículos. Se va acumulando el tiempo total invertido en revisión y, para cada artículo, se calcula el instante en el que quedaría completamente publicado sumando a ese acumulado su tiempo de maquetación. El tiempo total de la solución será el máximo de todos esos valores.

La idea del algoritmo sería: como la segunda fase puede ejecutarse en paralelo y lo único que retrasa su comienzo es la revisión, conviene lanzar cuanto antes a los artículos con tiempos de maquetación más largos. De este modo, mientras el comité sigue revisando otros artículos, esos trabajos más costosos ya estarán avanzando en paralelo en la fase de publicación.

Para ello hemos pensado este pseudocódigo:

```
Shell
```

```
GestionEditorialGreedy(Articulos):
```

```
    ordenar Articulos en orden no creciente según  $f(i)$ 
```

```
    tiempoRevision = 0
```

```
tiempoTotal = 0

para cada articulo i en Articulos:

    tiempoRevision = tiempoRevision + p(i)

    finPublicacion = tiempoRevision + f(i)

    si finPublicacion > tiempoTotal:

        tiempoTotal = finPublicacion

devolver Articulos, tiempoTotal
```

Este algoritmo devuelve por un lado el orden en el que se revisan los artículos y, por otro, el tiempo total necesario hasta que todos hayan quedado publicados.

1.3. Explicación del algoritmo

En primer lugar, se ordenan todos los artículos según su tiempo de maquetación $f(i)$ de mayor a menor. Esta ordenación es el núcleo de la decisión voraz, ya que fija qué artículos se consideran más prioritarios para pasar antes por la fase centralizada de revisión.

Después, el algoritmo recorre el vector ya ordenado una única vez. Se mantiene una variable acumuladora que representa el tiempo total consumido hasta ese momento por el comité editorial. Cada vez que se revisa un nuevo artículo, ese acumulado aumenta en $p(i)$.

En ese instante, el artículo ya puede entrar en maquetación, así que el momento en que terminará del todo es la suma del tiempo acumulado de revisión más su propio tiempo de maquetación $f(i)$.

Finalmente, el algoritmo va comparando esos tiempos de finalización y se queda con el mayor de todos, ya que el proceso completo no termina hasta que el último artículo quede publicado. Por eso, aunque cada artículo tenga su propio instante de finalización, el valor realmente importante es el máximo de todos ellos.

1.4. Análisis de eficiencia teórica

El coste dominante del algoritmo se encuentra en la ordenación inicial de los artículos según el valor $f(i)$. Si se utiliza una función de ordenación estándar, esta operación requiere un tiempo de $O(n \log(n))$, siendo n el número de artículos. Una vez realizada la ordenación, el algoritmo recorre el

vector exactamente una vez para acumular los tiempos de revisión y calcular el instante final de publicación de cada artículo, lo cual supone un coste lineal $O(n)$.

1.5. Demostración de optimalidad

En este tipo de problemas hay que justificar que la elección local conduce realmente a una solución global óptima. La demostración que utilizaremos aquí es un argumento de intercambio. La idea consiste en ver que, si en una planificación aparece antes un artículo con menor tiempo de maquetación que otro que viene después pero tiene mayor tiempo de maquetación, entonces intercambiar ambos no empeora la solución. Si este intercambio nunca perjudica, podremos repetirlo hasta obtener una secuencia ordenada de mayor a menor según $f(i)$, que será por tanto óptima.

1.6. Algoritmo alternativo de comparación

En nuestro caso, hemos elegido como alternativa una estrategia también intuitiva, aunque no óptima en general: ordenar los artículos por tiempo de revisión creciente, es decir, revisar primero los artículos que menos tarda en procesar el comité editorial.

Si se revisan primero los artículos más cortos, el comité editorial rápidamente muchos artículos, y eso podría parecer beneficioso porque permite que la fase de maquetación comience pronto en varios de ellos. Sin embargo, en este problema el criterio realmente importante no es cuánto tarda cada artículo en revisión, sino cuánto puede retrasar al instante final del proceso debido a su tiempo de maquetación posterior.

Pseudocódigo del algoritmo alternativo:

Shell

```
GestionEditorialAlternativo(Articulos):
```

```
    ordenar Articulos en orden creciente según p(i)
```

```
    tiempoRevision = 0
```

```
    tiempoTotal = 0
```

```
    para cada articulo i en Articulos:
```

```
        tiempoRevision = tiempoRevision + p(i)
```

```
        finPublicacion = tiempoRevision + f(i)
```

```
        si finPublicacion > tiempoTotal:
```

```
tiempoTotal = finPublicacion  
  
devolver Articulos, tiempoTotal
```

La única diferencia es el criterio de ordenación inicial: en lugar de priorizar los artículos con mayor tiempo de maquetación, prioriza los de revisión más corta.

1.6.1. Eficiencia teórica del algoritmo alternativo

Su análisis es igual al del algoritmo óptimo: la ordenación inicial cuesta $O(n \log(n))$ y el recorrido posterior del vector cuesta $O(n)$. Entonces, su orden de eficiencia teórica es $O(n \log n)$.

Dos algoritmos pueden tener la misma complejidad asintótica y, sin embargo, obtener soluciones de distinta calidad. En este caso, ambos algoritmos son igual de eficientes desde el punto de vista teórico, pero solo uno proporciona la solución óptima.

1.7. Implementación de los algoritmos

Para resolver el problema se han implementado dos algoritmos distintos. El primero de ellos es el algoritmo voraz óptimo, que ordena los artículos en orden no creciente según su tiempo de maquetación $f(i)$. El segundo es un algoritmo alternativo, que ordena los artículos en orden creciente según su tiempo de revisión $p(i)$. En ambos casos, una vez fijado el orden, se recorre la secuencia acumulando los tiempos de revisión y calculando el instante en el que cada artículo termina completamente su proceso de publicación. El valor final de la solución es el máximo de dichos instantes.

Código del algoritmo voraz óptimo:

C/C++

```
struct Articulo {  
    int id;  
  
    int p; // tiempo de revisión  
  
    int f; // tiempo de maquetación  
};  
  
struct Resultado {
```

```
vector<Articulo> orden;

long long tiempoTotal;

};

Resultado gestionEditorialGreedy(vector<Articulo> articulos) {
    sort(articulos.begin(), articulos.end(), [](const Articulo
&a, const Articulo &b) {
        if (a.f != b.f) return a.f > b.f; // mayor f primero
        return a.id < b.id; // desempate estable
    });

    long long tiempoRevisionAcumulado = 0;

    long long tiempoTotal = 0;

    for (int i = 0; i < (int)articulos.size(); i++) {
        tiempoRevisionAcumulado += articulos[i].p;

        long long finPublicacion = tiempoRevisionAcumulado +
articulos[i].f;

        if (finPublicacion > tiempoTotal)
            tiempoTotal = finPublicacion;
    }

    Resultado res;

    res.orden = articulos;

    res.tiempoTotal = tiempoTotal;

    return res;
}
```

La implementación del algoritmo voraz comienza ordenando el vector de artículos según el valor $f(i)$ de mayor a menor. Esta parte es la decisión greedy del problema, ya que da prioridad a los artículos cuya fase de maquetación es más larga. Una vez fijado ese orden, el algoritmo recorre el vector acumulando el tiempo de revisión. Para cada artículo calcula su instante de finalización completa, sumando al tiempo de revisión acumulado su tiempo de maquetación. El resultado final es el mayor de esos valores, ya que el proceso editorial solo termina cuando el último artículo queda publicado.

Código del algoritmo alternativo:

C/C++

```
Resultado gestionEditorialAlternativo(vector<Articulo> articulos)
{
    sort(articulos.begin(), articulos.end(), [](const Articulo
&a, const Articulo &b) {
        if (a.p != b.p) return a.p < b.p; // menor p primero
        return a.id < b.id; // desempate estable
    });

    long long tiempoRevisionAcumulado = 0;
    long long tiempoTotal = 0;
    for (int i = 0; i < (int)articulos.size(); i++) {
        tiempoRevisionAcumulado += articulos[i].p;

        long long finPublicacion = tiempoRevisionAcumulado +
        articulos[i].f;

        if (finPublicacion > tiempoTotal)
            tiempoTotal = finPublicacion;
    }

    Resultado res;
```

```
    res.orden = articulos;

    res.tiempoTotal = tiempoTotal;

    return res;
}
```

Este segundo algoritmo utiliza la misma mecánica de cálculo que el anterior. La única diferencia está en el criterio de ordenación inicial, en lugar de priorizar los artículos con mayor tiempo de maquetación, prioriza los que tienen menor tiempo de revisión.

Función para mostrar resultados:

C/C++

```
void mostrarResultado(const string &nombre, const Resultado &res)
{
    cout << nombre << endl;

    cout << "Orden de revisión: ";

    for (int i = 0; i < (int)res.orden.size(); i++) {
        cout << res.orden[i].id;

        if (i + 1 < (int)res.orden.size()) cout << " - ";
    }

    cout << endl;

    cout << "Tiempo total: " << res.tiempoTotal << endl;
}
```

Ambas implementaciones son similares porque el problema no está en el cálculo del tiempo final, sino en el criterio de ordenación. Vemos como con una estructura casi idéntica y la misma eficiencia teórica, se obtienen soluciones de distinta calidad dependiendo de cómo se elija el orden inicial de los artículos.

1.8. Análisis de eficiencia empírica

Una vez implementados los dos algoritmos, se ha realizado un estudio empírico con el objetivo de comparar su comportamiento práctico.

Para realizar este estudio, se han generado distintas instancias del problema con un número creciente de artículos. En cada instancia, a cada artículo se le han asignado dos valores enteros positivos: un tiempo de revisión $p(i)$ y un tiempo de maquetación $f(i)$. Posteriormente, se han ejecutado sobre esas mismas instancias tanto el algoritmo voraz óptimo como el algoritmo alternativo, midiendo en ambos casos el tiempo de ejecución y el tiempo total de publicación obtenido.

Dado que los dos algoritmos comparten prácticamente la misma estructura y solo difieren en el criterio de ordenación inicial, es esperable que sus tiempos de ejecución sean parecidos. En ambos casos, la operación dominante sigue siendo la ordenación de los artículos, que requiere un tiempo del orden $O(n \log n)$. Por ello, la comparación experimental resulta más interesante desde el punto de vista de la calidad de la solución que desde el tiempo de ejecución.

La experimentación se ha realizado haciendo crecer el tamaño de entrada N , entendiendo por N el número de artículos de cada instancia. Para cada tamaño se ha ejecutado ambos algoritmos sobre la misma entrada, registrando:

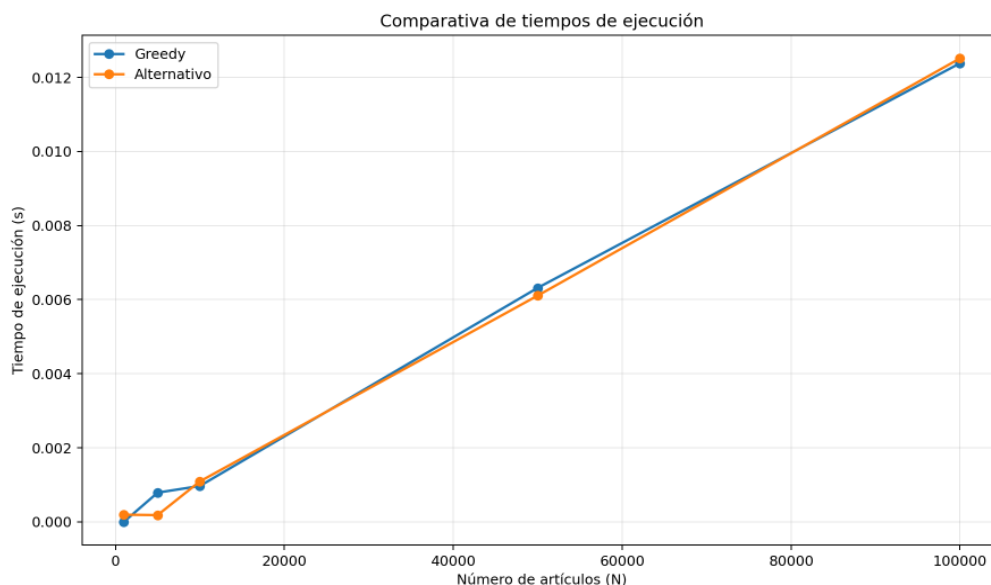
- el número de artículos
- el tiempo de ejecución del algoritmo voraz óptimo
- el tiempo de ejecución del algoritmo alternativo
- el tiempo total de publicación del algoritmo voraz óptimo
- y el tiempo total de publicación del algoritmo alternativo.

De este modo, puede observarse no sólo cómo crece el coste computacional de ambos métodos, sino también cómo se comporta cada uno en términos de calidad de solución.

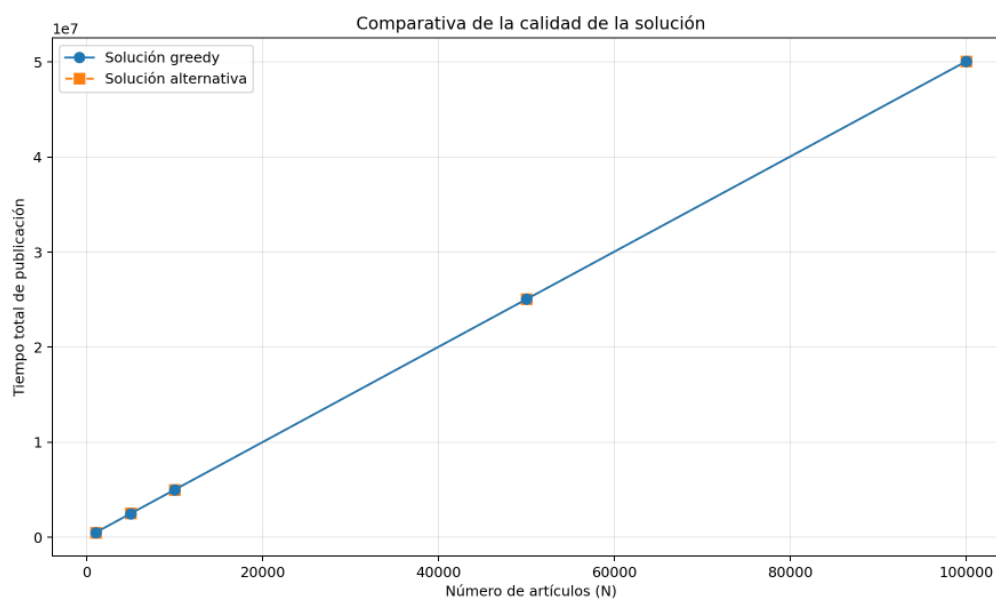
N	T.greedy	T.alternativo	Sol.greedy	Sol.alternativa
1000	0.00000000	0.00019380	497337.200	497855.600
5000	0.00078920	0.00018320	2503144.40	2503893.800
10000	0.00096920	0.00109160	5004469.20	5004994.600
50000	0.00631660	0.00610800	25048263.20	25048686.400
100000	0.01238080	0.01251120	50065950.80	50066411.000

Se observa que ambos algoritmos presentan tiempos de ejecución muy similares, ya que en los dos casos el coste dominante es la ordenación inicial. Vemos cómo el algoritmo voraz óptimo obtiene siempre una solución mejor o igual que la del algoritmo alternativo.

Gráfica de tiempos de ejecución:



Gráfica de calidad de la solución:



2. Cámaras de vigilancia

2.1. Explicación del Diseño y Análisis del Algoritmo Voraz

Para resolver este problema mediante un algoritmo voraz, me inspiré inicialmente en la estructura utilizada por el enfoque de fuerza bruta. Al ver que empleaban una matriz de adyacencia para representar las intersecciones y los pasillos, decidí utilizar esa misma estructura como base de mi lógica.

Mi planteamiento fue el siguiente: si queremos minimizar el número de cámaras, la mejor heurística es colocar la cámara en la intersección que cubra la mayor cantidad de pasillos no vigilados. Para implementarlo, el algoritmo recorre la matriz contando cuántos unos (pasillos) tiene cada fila (intersección). Seleccionamos siempre la intersección con el mayor número de pasillos conectados a ella.

Una vez elegida la intersección, es crucial no volver a contar los pasillos que ya han sido vigilados. Por ello, elimino esos pasillos recorriendo la fila y la columna seleccionada y llenándolas de ceros. De esta forma, si otra intersección compartía pasillos con la que acabamos de cubrir, su número de pasillos se actualiza correctamente. Este proceso se repite en bucle hasta que ya no quede ningún 1 en la matriz, lo que significa que todos los pasillos están vigilados.

2.2. Análisis de las partes relevantes del código

```
for (int i = 0; i < n; i++) {
    int sumaPasillos = 0;
    for (int j = 0; j < n; j++) {
        if (matriz[i][j] == 1) {
            sumaPasillos++;
        }
    }

    if (sumaPasillos > maxPasillos) {
        maxPasillos = sumaPasillos;
        mejorInterseccion = i;
    }
}
```

Este bucle anidado es el corazón del problema. En cada iteración, el algoritmo necesita decidir dónde poner la siguiente cámara. Para ello, recorre cada fila de la matriz y cuenta cuántos unos (pasillos sin vigilar) tiene conectados.

Si la intersección actual tiene más pasillos sin vigilar que el máximo registrado hasta el momento ($\text{sumaPasillos} > \text{maxPasillos}$), se actualiza como la nueva candidata. Aquí es exactamente donde el algoritmo toma su decisión basada en el óptimo local: selecciona ciegamente la opción que mayor beneficio inmediato aporta en ese instante preciso, sin evaluar cómo le va a afectar esta decisión en el futuro.

```
pasillosConCamaras.push_back(mejorInterseccion + 1);
camarasUsadas++;
```

```
for (int j = 0; j < n; j++) {  
    matriz[mejorInterseccion][j] = 0;  
    matriz[j][mejorInterseccion] = 0;  
}
```

Un detalle de implementación a destacar es el +1 al guardar la mejorInterseccion. Dado que los índices de la matriz en C++ empiezan en 0, realizo este ajuste para que en la salida por pantalla las intersecciones se numeren desde el uno.

Además, en esta parte del código encontramos el bucle encargado de la limpieza. Al colocar una cámara, todos los pasillos que llegan a esa intersección quedan vigilados. Por tanto, es estrictamente necesario poner a '0' todos los valores de su fila y su columna. Si no hiciéramos esto, en la siguiente iteración el algoritmo volvería a contar pasillos que ya están vigilados, tomando decisiones erróneas o entrando en un bucle infinito.

2.3. Casos subóptimos: Dónde y por qué falla el algoritmo

Tras implementar esta lógica, estuve realizando pruebas con matrices generadas aleatoriamente. En la inmensa mayoría de los casos, el algoritmo se comportó muy bien, ofreciendo soluciones rápidas y aparentemente óptimas.

Sin embargo, al analizar la heurística, me di cuenta de que su naturaleza de buscar siempre el beneficio inmediato la hacía vulnerable. Para probar esto, diseñe varias matrices a mano enfrentando los resultados de mi programa contra el de fuerza bruta que me asegura encontrar la mejor solución sacrificando el tiempo.

2.3.1. Primer fallo

Lo primero que se me ocurrió fue pensar en situaciones donde varias intersecciones tienen exactamente la misma cantidad máxima de pasillos. Debido a la condición `if (sumaPasillos > maxPasillos)`, mi algoritmo siempre se queda con el primer máximo que encuentra y vi la posibilidad de forzar un fallo.

```
0 1 1 0 0 0 0  
1 0 0 0 0 0 1  
1 0 0 1 0 0 0  
0 0 1 0 1 0 0  
0 0 0 1 0 1 0  
0 0 0 0 1 0 0  
0 1 0 0 0 0 0
```

Solución Óptima (Fuerza Bruta): 3 cámaras (Intersecciones 2, 3 y 5)

Solución Algoritmo Voraz: 4 cámaras (Intersecciones 1, 4, 2 y 5)

En esta matriz, los nodos del 1 al 5 tienen exactamente 2 pasillos (un empate quintuple). Al elegir ciegamente el Nodo 1 solo por ser el primero del bucle for, el algoritmo "parte" el camino en dos fragmentos sueltos. Esto le obliga a usar 4 cámaras en total para parchear el error, mientras que la fuerza bruta selecciona estratégicamente los nodos alternos logrando vigilar todo con solo 3 cámaras.

2.3.2. Segundo fallo

Para forzar otro error, construí una matriz de 7x7 que simula una forma de estrella: un nodo central conectado a tres nodos intermedios, que a su vez daban a tres callejones sin salida. Efectivamente, el algoritmo falló. Fue engañado por el "nodo cebo" central (que tenía 3 pasillos). Al elegir el centro primero, desconectó el resto de la red, dejando los tres callejones aislados.

```
0 1 1 1 0 0 0
1 0 0 0 1 0 0
1 0 0 0 0 1 0
1 0 0 0 0 0 1
0 1 0 0 0 0 0
0 0 1 0 0 0 0
0 0 0 1 0 0 0
```

Solución Óptima (Fuerza Bruta): 3 cámaras (Intersecciones 2, 3 y 4)

Solución Algoritmo Voraz: 4 cámaras (Intersecciones 1, 2, 3 y 4)

El algoritmo no puede resistirse al Nodo 1 porque es el máximo absoluto (3 conexiones). Sin embargo, la verdadera solución óptima consistía en ignorar el centro y colocar cámaras solo en los 3 nodos intermedios, ya que estos cubrían simultáneamente el centro y los callejones exteriores

2.3.3. Tercer fallo

Finalmente, busqué una matriz más rebuscada para forzar un fallo superior a una cámara. Lo hice con dos nodos cebo de grado 4, rodeados de nodos intermedios de grado 3.

```
0 0 1 1 1 1 0 0 0 0
0 0 1 1 1 1 0 0 0 0
1 1 0 0 0 0 1 0 0 0
1 1 0 0 0 0 0 1 0 0
1 1 0 0 0 0 0 0 1 0
1 1 0 0 0 0 0 0 0 1
0 0 1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
```

0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0

Solución Óptima (Fuerza Bruta): 4 cámaras (Intersecciones 3, 4, 5 y 6)

Solución Algoritmo Voraz: 6 cámaras (Intersecciones 1, 2, 3, 4, 5 y 6)

Al evaluar, el algoritmo voraz tropezó dos veces consecutivas: eligió el primer nodo de grado 4 y luego el segundo de grado 4. Estas elecciones, aislaron 4 aristas en los extremos y forzaron al algoritmo a gastar 4 cámaras adicionales (6 cámaras en total). La fuerza bruta demostró que ignorando estos, todo podía vigilarse utilizando únicamente 4 cámaras.

2.4. Eficiencia Teórica

Vamos a analizar paso a paso la eficiencia de dicho algoritmo. Partimos de un bucle while, pero lo voy a dejar para lo último, dentro de dicho bucle, pasadas unas declaraciones, tenemos un bucle anidado, ambos recorren el tamaño entero por lo que nos encontramos con un orden de eficiencia de $O(n^2)$. A continuación, tenemos comparaciones y asignaciones que tienen un orden de eficiencia de $O(1)$, hasta que llegamos a otro bucle que recorrerá también todo el tamaño en filas, mientras que haya pasillos inicializados a 1, este orden de eficiencia es de $O(n)$.

Ahora vamos con el bucle while, este bucle se repetirá tantas veces como cámaras haya que usar, por tanto vamos a llamar k al número de cámaras que se han colocado, por tanto el orden de eficiencia $O(n^2)$ le vamos a añadir k , ya que dependiendo de k puede ser un mejor o peor caso.

El tiempo de ejecución de este algoritmo es el siguiente:

$$T(n, k) = O(k(n^2 + n))$$

Al ser $O(n^2)$ el término dominante, este algoritmo tiene un orden de eficiencia de $O(kn^2)$.

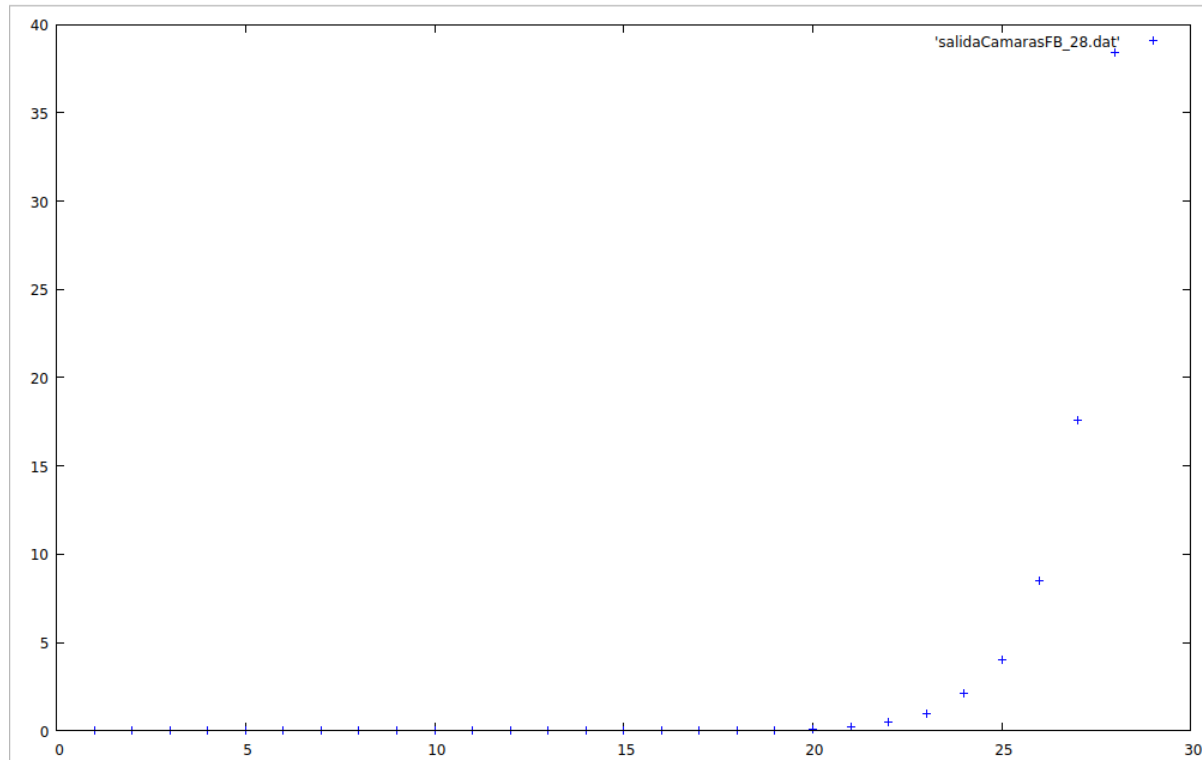
En el mejor de los casos ($k = 1$, una sola cámara cubre todas las intersecciones) el orden de eficiencia sería $O(n^2)$. Y en el peor de los casos ($k = n$, se necesita una cámara en cada intersección) el orden de eficiencia sería $O(n^3)$.

2.5. Eficiencia Empírica

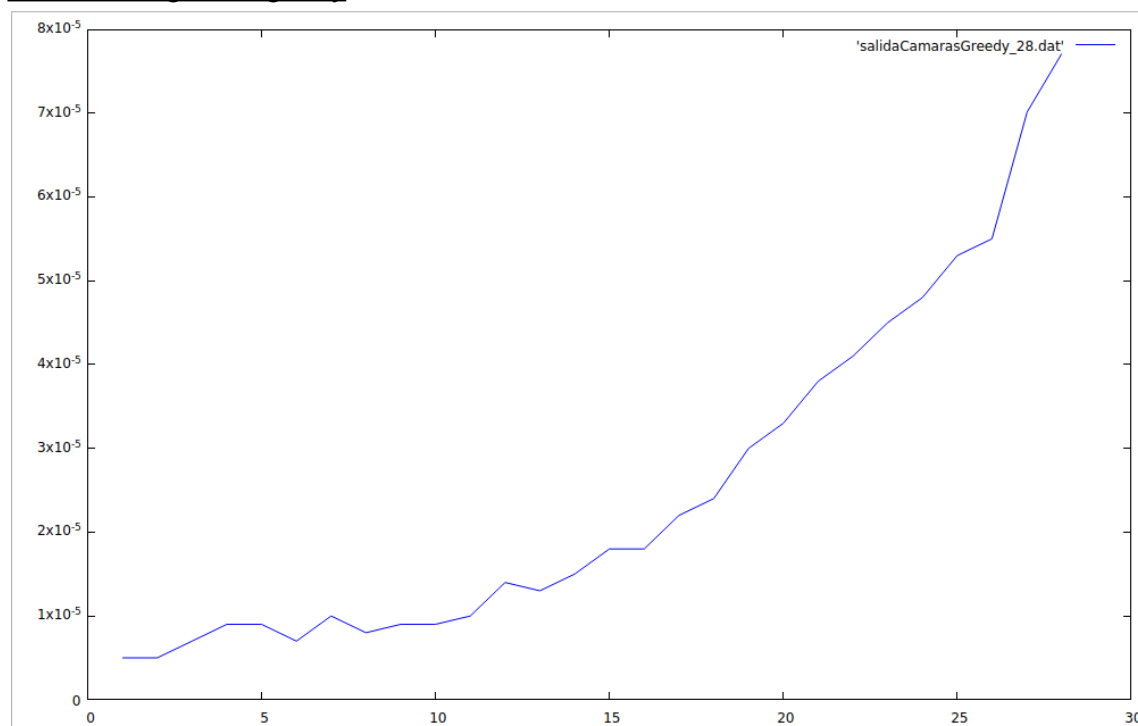
Para calcular la eficiencia empírica he probado con distintos tamaños, no solo para comparar el tiempo de ejecución si no para saber el número de cámaras que se usan en cada caso, vamos a analizarlos uno por uno.

Tamaño 28

Gráfica del algoritmo fuerza bruta



Gráfica del algoritmo greedy



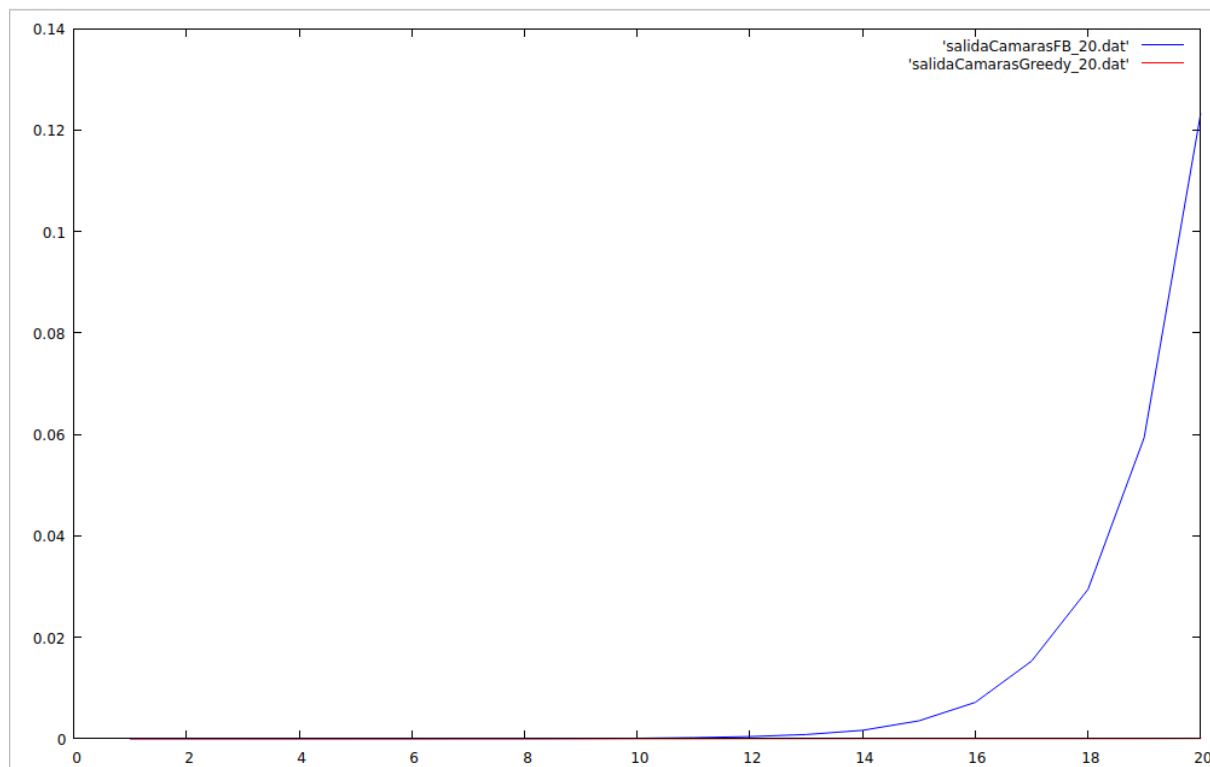
Las he puesto por separado ya que si las pusiéramos juntas en una misma gráfica, la línea correspondiente al algoritmo greedy no aparecería. Fijándonos en los tiempos de ejecución de cada algoritmo podemos observar que hay una gran diferencia, el de fuerza bruta tarda aproximadamente unos 37 segundos y en cambio greedy no llega al segundo.

Pero vamos a comparar las cámaras que ha usado cada algoritmo:

El algoritmo de fuerza bruta ha usado un total de 20 cámaras, al igual que el algoritmo de greedy, en este caso es más efectivo el algoritmo greedy porque a pesar de que han usado las mismas cámaras tarda bastante tiempo menos.

Tamaño 20

Gráfica de ambos algoritmos



Con este tamaño podemos observar que pasa como el tamaño anterior, aunque se puede apreciar un poquito más la línea correspondiente al algoritmo greedy. A pesar de que en la gráfica hay bastante diferencia, aunque realmente la hay, el algoritmo de fuerza bruta no llega al segundo, es decir, tarda bastante poco, el algoritmo greedy tarda unos 0.071 milisegundos, más rápido aún.

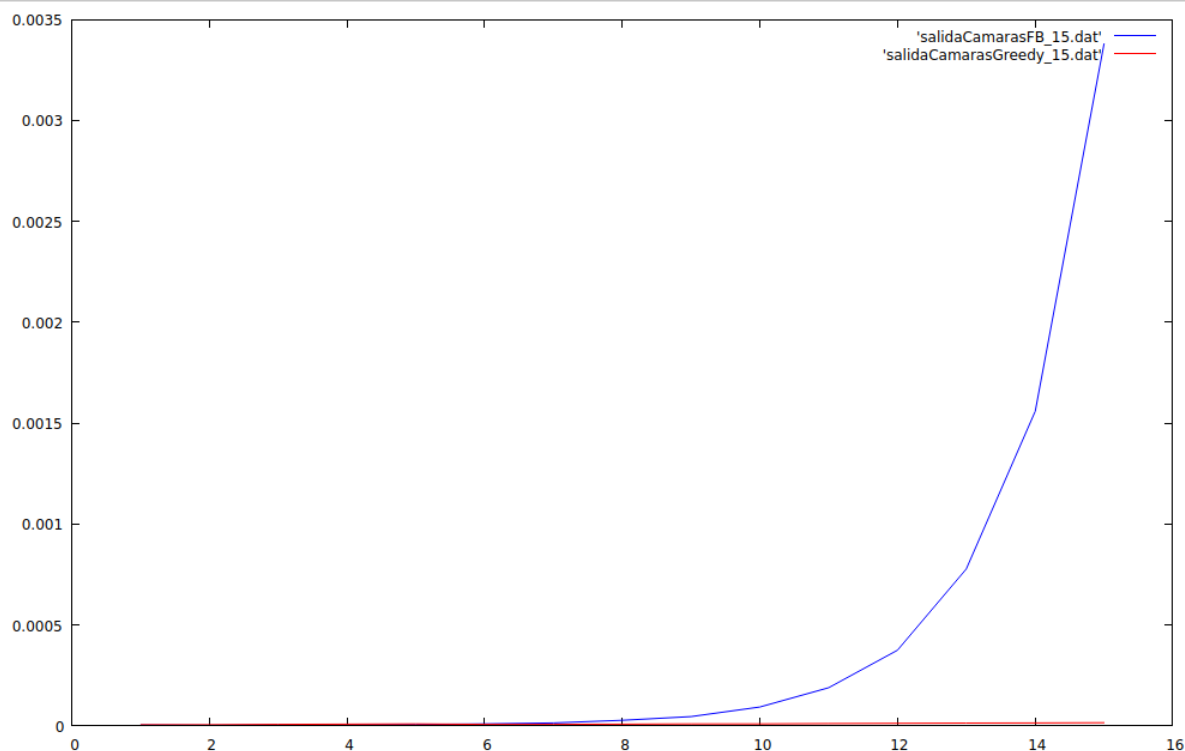
En cuanto al número de cámaras que ha usado cada algoritmo ha sido el siguiente:

El algoritmo de fuerza bruta ha usado un total de 11 cámaras y el algoritmo greedy un total de 14 cámaras.

En este caso, el algoritmo de fuerza bruta es más efectivo, a pesar de que tarda más que el algoritmo greedy (realmente tarda relativamente muy poco, no llega ni al medio segundo), ya que usa menos cámaras.

Tamaño 15

Gráfica de ambos algoritmos



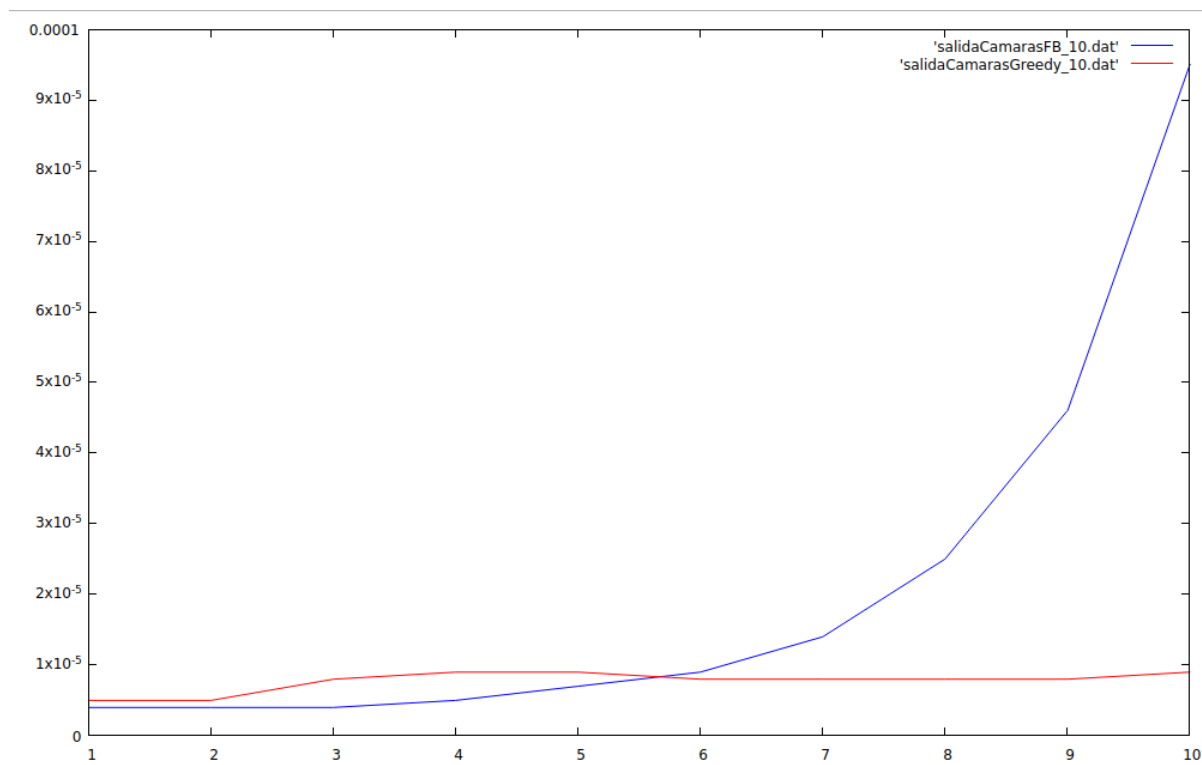
Este caso es semejante al anterior, ambos tienen un tiempo de ejecución muy bajo, aunque el algoritmo greedy más bajo aún, pero ninguno llega al segundo.

En cuanto al número de cámaras que ha usado cada uno es el siguiente:

El algoritmo de fuerza bruta ha usado 9 cámaras y greedy ha usado 10, nos encontramos con el mismo caso que el anterior, aunque la diferencia es de una cámara el algoritmo de fuerza bruta nos demuestra que ha encontrado una solución más óptima que greedy.

Tamaño 10

Gráfica de ambos algoritmos



Aquí ya podemos empezar a apreciar mejor la gráfica, greedy sigue tardando menos, como es obvio menor sea el tamaño menos tardan ambos algoritmos. En este caso el número de cámaras que usado cada algoritmo ha sido el siguiente:

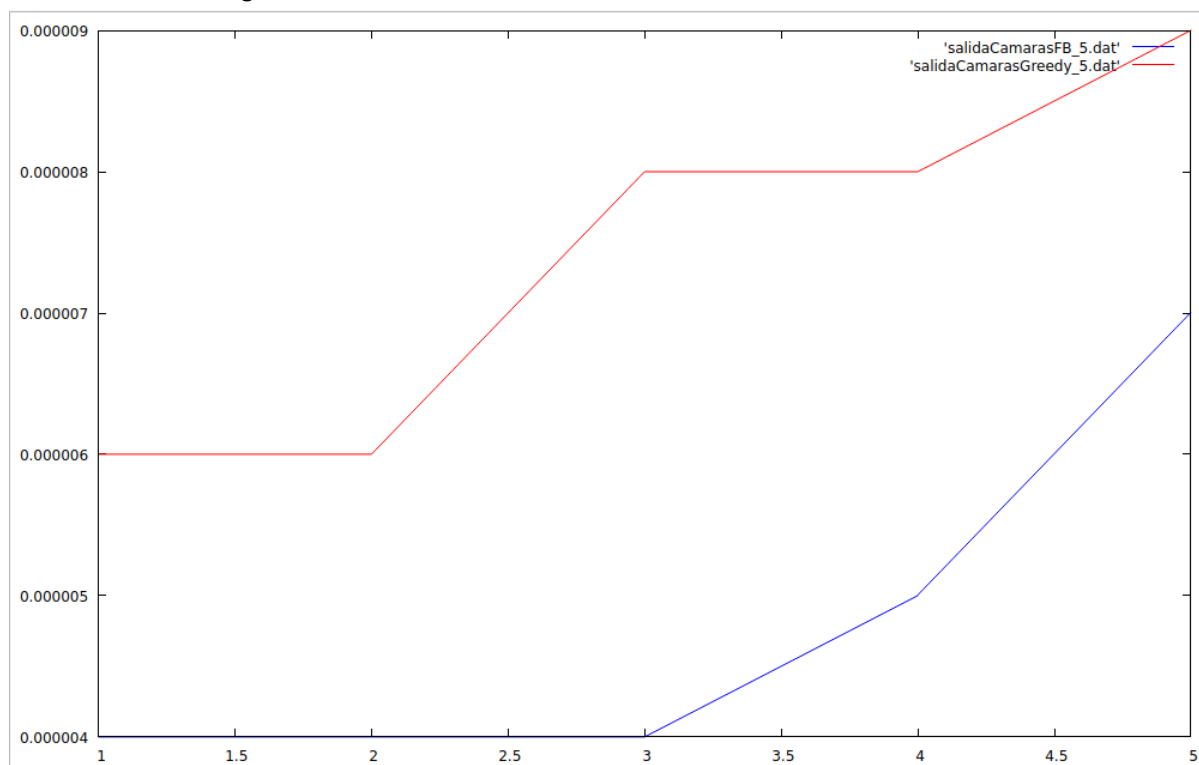
Greedy ha usado 4 cámaras y el de fuerza bruta 5.

Aquí sí podemos decir que es más efectivo greedy, tanto en tiempo de ejecución como en número de cámaras usadas.

También he probado a hacerlo con tamaño 8, no añado captura de la gráfica ya que se repite el mismo caso que con el tamaño 10, pero con este tamaño el tiempo de ejecución de ambos algoritmos se empieza a parecer más y a ser un poco más igualatorios.

Tamaño 5

Gráfica de ambos algoritmos



Con este tamaño podemos observar que el algoritmo greedy tarda más que el algoritmo de fuerza bruta.

Las cámaras que han usado ambos algoritmos han sido:

El algoritmo de fuerza bruta ha usado 1 cámara, en cambio greedy ha usado 2.

Aquí nuevamente el algoritmo de fuerza bruta es más efectivo, y en este caso tanto en tiempo de ejecución como en número de cámaras.

Conclusión

En general, el algoritmo Greedy tiene menor coste que fuerza bruta, ya que este prueba todas las posibilidades posibles, en cambio greedy no prueba todas las posibilidades, el problema de este enfoque es que no siempre va a encontrar la solución óptima, y para tamaños pequeños, en algunas ocasiones, va a ser mejor usar el algoritmo de fuerza bruta.

El problema de hacerlo por fuerza bruta es que a medida que el tamaño crece esta opción no es viable, siendo preferible utilizar Greedy aunque no encuentre la solución óptima.

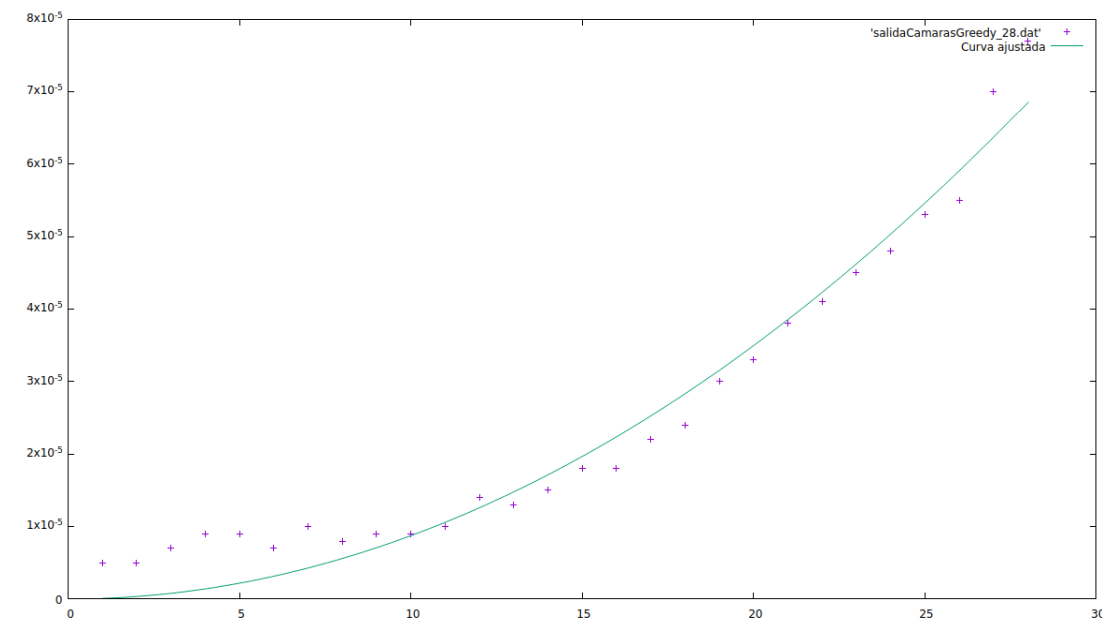
2.6. Eficiencia Híbrida

Para calcular la constante oculta he probado con $f(x)$ en el mejor y peor de los casos.

Para $f(x) = a_0 * x * x \rightarrow O(n^2)$

Final set of parameters		Asymptotic Standard Error	
=====		=====	
a0	= 8.73856e-08	+/- 2.113e-09	(2.418%)

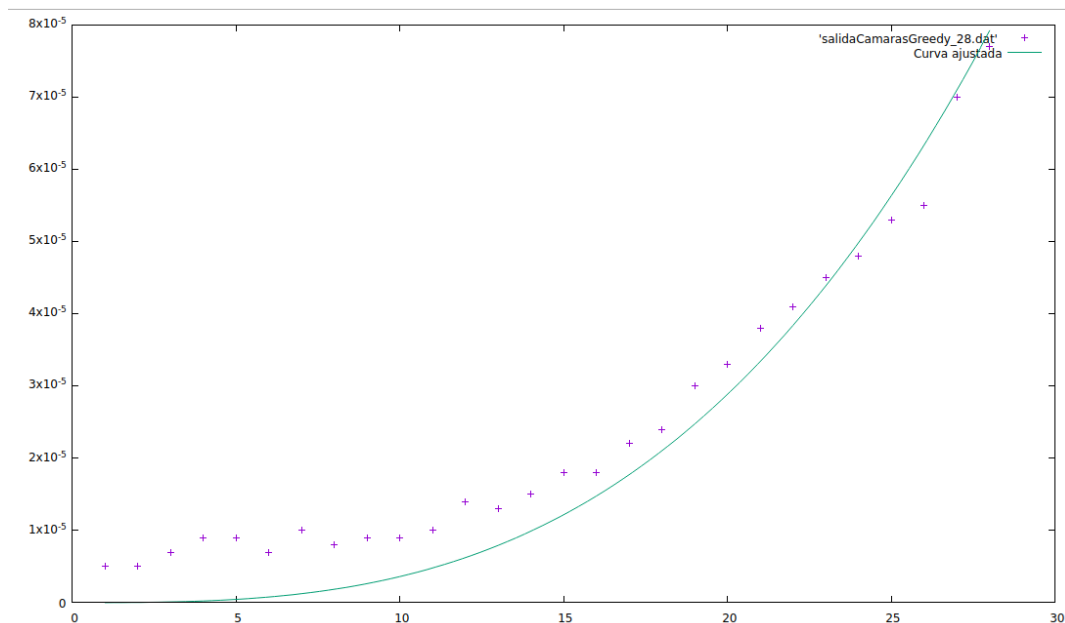
Gráfica correspondiente



Para $f(x) = a_0 * x * x * x \rightarrow O(n^3)$

Final set of parameters		Asymptotic Standard Error	
=====		=====	
a0	= 3.60503e-09	+/- 1.2e-10	(3.327%)

Gráfica correspondiente



Como podemos observar la eficiencia del algoritmo no se ajusta del todo a la curva teórica, ya que depende no solo del tamaño del problema (n), sino también del número de cámaras utilizadas (k).

En el mejor de los casos, donde $k = 1$, si se ajustaría a la curva, lo mismo pasa en el peor de los casos, donde $k = n$, se ajustaría a su curva correspondiente.

Por tanto, la desviación respecto a las curvas teóricas se debe a que el algoritmo no presenta un único comportamiento, sino que depende del número de cámaras usadas.

3. Cámaras en árbol (opcional)

3.1. Diferencias en complejidad

El problema de las cámaras original, sobre un grafo, se trata de un problema conocido como Minimum Vertex Cover y es un problema NP-Completo (comprobar si la solución es correcta es fácil y no se conoce algoritmo polinomial que lo resuelva).

En un grafo general, cuando decides poner o no una cámara en un nodo, esa decisión afecta a vecinos que a su vez están conectados entre sí de formas complejas (puede haber ciclos y atajos a caminos).

No puedes tomar decisiones locales sin mirar el panorama completo del grafo.

En un árbol en cambio, si miras una arista padre-hijo, esa arista solo puede ser cubierta por esos dos nodos y ninguno más. No hay otro camino alternativo. Esto significa que puedes tomar decisiones de abajo hacia arriba, subárbol por subárbol, sin que una decisión en una rama afecte jamás a otra rama. Todo esto viene a decir que al ser un árbol el problema implícitamente se vuelve significativamente más sencillo.

Veremos ahora cómo el diseño de un algoritmo voraz que resuelve este problema resulta demostrar como de significativa esa reducción de dificultad.

3.2. Diseño del algoritmo

3.2.1. Explicación de la estrategia

La principal variable de la resolución del problema es ¿qué posición de una cámara garantiza ser la mejor opción? Al tener estructura de un árbol podemos descartar un subconjunto claro de nodos que son subóptimos: las hojas. De poner una cámara sobre una hoja cubrimos el pasillo padre-hijo, pero dicho padre tiene (salvo que sea raíz) otro padre y por tanto otro pasillo que solo puede ser cubierto por él o por el padre de este. Es decir, si nunca seleccionamos una hoja, pero sí su padre, estamos cubriendo de forma óptima los pasillos que dan a las hojas.

De dicha lógica podemos pensar en la siguiente estrategia para resolver el problema: En cada paso, tomamos cualquier hoja del árbol actual, colocamos la cámara en su padre, y cubrimos así todos los pasillos que llegan a dicho padre. Una vez hecho esto, los nodos que queden completamente aislados (sin pasillos sin cubrir) se eliminan del árbol, lo que puede generar nuevas hojas sobre las que repetir el proceso. El algoritmo termina cuando todos los pasillos están cubiertos.

La estrategia es óptima ya que nunca se desperdicia una cámara: al colocarla siempre en el padre de una hoja, garantizamos que cubre como mínimo el pasillo hacia esa hoja y potencialmente varios pasillos más, mientras que colocarla en la hoja solo cubriría ese único

pasillo. Por tanto, el padre es siempre igual de bueno o mejor que la hoja, y nunca necesitaremos volver atrás para reconsiderar una decisión tomada.

3.2.2. Pseudocódigo

Shell

CAMARAS-ARBOL(T):

camaras $\leftarrow \{\}$

mientras existan pasillos sin cubrir en T:

u $\leftarrow$ cualquier hoja de T

p $\leftarrow$ padre de u

camaras $\leftarrow$ camaras $\cup \{p\}$

MARCAR COMO CUBIERTOS todos los pasillos adyacentes a p

ELIMINAR de T los nodos que hayan quedado aislados

devolver camaras

3.3. Conclusiones

Es sencillo ver cómo el algoritmo implementado, no solo encuentra una solución óptima para el problema sino que además tiene una eficiencia de orden

$O(n + m) = O(n)$, tal que $n = n^{\circ}$ vértices y $m = n^{\circ}$ pasillos del árbol.

Esto significa que al cambiar de un grafo cualquiera a un árbol, el problema opcional es de complejidad P porque existe un algoritmo que lo resuelve en tiempo polinomial (de hecho es lineal: $O(n)$). Esto contrasta fuertemente con el caso general que es NP-Complete.

La razón de esta diferencia la comentamos al principio: la ausencia de ciclos en un árbol permite que las decisiones óptimas locales que sean también óptimas globales y por tanto el algoritmo greedy funciona perfectamente.

La conclusión que sacamos de esto es el cómo pequeñas variaciones a las restricciones de un problema pueden cambiar enormemente la complejidad del mismo.